2048 as a Markov Decision Problem: exact dynamic programming on small boards and reinforcement learning on the real game

Project work — Dynamic Programming and Reinforcement Learning, A.Y. 2024/25

Edoardo Caproni ([matricola]) Tommaso [Surname] ([matricola])

November 4, 2026

Abstract

2048 is a single-player sliding-tile puzzle whose rules define a Markov decision problem with a known model. We solve small versions of the game exactly by dynamic programming, exploiting the fact that the sum of the tiles is a clock that turns the problem into a finite-horizon one, and we use the exact solution to measure how far the usual rules of thumb are from optimal play and how much of the outcome is luck. We then drop the knowledge of the model and learn to play from a simulator: tabular Monte Carlo and temporal-difference control on a board whose optimal values are known, and temporal-difference learning of an n-tuple network on afterstates on the original 4×4 board, where the state space ($\approx 10^{16}$ states) rules out exact methods. The exact values validate every learning algorithm. We close with what is known about the game itself: whether it can always be won, whether an optimal strategy is known, and which heuristics are good.

1 Motivation

2048 (G. Cirulli, 2014 [3]) is a single-player puzzle played on a 4×4 grid of numbered tiles. Every move is a deterministic slide of all the tiles followed by the random appearance of a new tile, so the game is a Markov decision problem whose model is completely known. Small versions of the game (fewer cells, a smaller target tile) can be solved exactly by dynamic programming. The original game cannot: its state space is far too large. The project is built on this gap.

- **DP part.** On small boards, compute the optimal policy exactly and use it to answer questions that are hard to settle by playing: how well can the game be played, how good are the rules of thumb used by human players, and how much of the outcome is luck.
- **RL part.** Remove the knowledge of the model and move to the original 4×4 board, where only learning with function approximation is feasible. The exact solutions of the DP part validate the learning algorithms.

2 The game

The game is played on a board of $R \times C$ cells with a target tile 2^K. In the original game $R = C = 4$, $K = 11$ (tile 2048) and $p_4 = 0.1$.

1. A move $u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\}$ slides all the tiles as far as possible in that direction. Two tiles with the same value that collide merge into one tile carrying their sum. A tile created by a merge cannot merge again in the same move, and in each line the merges are resolved starting from the side the tiles move to: moving left, 2 2 2 2 becomes 4 4. The score increases by the value of every tile created by a merge.

2. A move is legal only if it changes the board.

3. After every legal move a new tile appears in an empty cell chosen uniformly at random: a 2 with probability $1 - p_4$, a 4 with probability p_4. The game starts with two such tiles in two distinct cells.

4. The game is won when a move creates the target tile, and lost when no legal move exists.

3 Formulation as a Markov decision problem

State. A board is encoded by the exponents of its tiles, $x = (x_1, \dots, x_{RC})$ in row-major order, with $x_i = 0$ for an empty cell and $x_i = k$ for a tile 2^k. The state x_t is the board observed before the t-th move, that is, after the random tile. The initial state x_0 has distribution p_0: two tiles in two distinct cells chosen uniformly at random, each a 2 with probability $1 - p_4$ or a 4 with probability p_4. Since the game ends as soon as the target is created, $x_i \in \{0, \dots, K-1\}$, and the state space $\mathcal{X}$ is the set of boards reachable from p_0. Two kinds of terminal situations exist: the win, reached when a move creates 2^K, and the boards with no legal move (game over).

Control. The admissible controls are the legal moves,

$$\mathcal{U}(x) = \{u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\} : M(x,u) \neq x\},$$

where M is the slide/merge map defined below; the game is over exactly when $\mathcal{U}(x) = \emptyset$.

Deterministic part of a move. $y = M(x,u)$ is the board after the slide/merge and before the new tile, called the *afterstate*. M acts line by line: each row (for left/right) or column (for up/down), read towards the side of the move, is compacted by removing the empty cells, adjacent equal pairs are merged from the front with each tile used at most once, and the line is padded with empty cells. The points of the move, $r(x,u)$, are the sum of the values of the merged tiles.

Disturbance. $w_t = (i_t, v_t)$, where i_t is uniform over the set $E(y_t)$ of empty cells of the afterstate $y_t = M(x_t, u_t)$, and $v_t \in \{1,2\}$ is the exponent of the new tile, with $P(v_t = 1) = 1 - p_4$ and $P(v_t = 2) = p_4$. Given y_t, w_t is independent of the past; its distribution depends on (x_t, u_t) only through y_t.

Dynamics. Writing $y + v\, e_i$ for the board y with exponent v placed in the empty cell i,

$$x_{t+1} = f(x_t, u_t, w_t) = M(x_t, u_t) + v_t\, e_{i_t}.$$

Objective. Maximise the probability of creating the target tile,

$$V^*(x) = \max_{\pi}\ P^{\pi}(\text{the target tile is created} \mid x_0 = x).$$

In the cost form used in the course [2], this is a problem with no stage cost and a terminal cost equal to 1 for a lost game and 0 for a won game; its optimal cost is $J^*(x) = 1 - V^*(x)$, the minimum probability of losing. A second objective is studied for comparison: the expected score, with stage reward $r(x_t, u_t)$ and no stop at the target tile.

Bellman equation. With the afterstate value W^*,

$$V^*(x) = \begin{cases} 0 & \text{if } \mathcal{U}(x) = \emptyset, \\ \max\limits_{u \in \mathcal{U}(x)} W^*\big(M(x,u)\big) & \text{otherwise,} \end{cases} \tag{1}$$

$$W^*(y) = \begin{cases} 1 & \text{if } \max_i y_i = K, \\ \displaystyle\sum_{i \in E(y)} \frac{1}{|E(y)|} \Big[(1-p_4)\, V^*(y+e_i) + p_4\, V^*(y+2e_i)\Big] & \text{otherwise.} \end{cases} \tag{2}$$

For the expected score, the maximisation becomes $\max_u \big[r(x,u) + W(M(x,u))\big]$ and the first case of (2) disappears.

4 Structural properties

Proposition 1 (the tile sum is a clock)**.** *Let $\sigma(x) = \sum_{i:\, x_i > 0} 2^{x_i}$ be the sum of the tiles. For every board x, legal move u and disturbance $w = (i, v)$,*

$$\sigma\big(M(x,u)\big) = \sigma(x), \qquad \sigma\big(f(x,u,w)\big) = \sigma(x) + 2^v \in \{\sigma(x)+2,\ \sigma(x)+4\}.$$

Proof. Sliding moves tiles without changing their values, and a merge replaces two tiles 2^k with one tile $2^{k+1} = 2^k + 2^k$. The new tile adds 2^v. □

Corollary 1. *(i) The tile sum strictly increases along every trajectory, so no state is visited twice: the transition graph of the reachable states is acyclic.*

(ii) Before the target is created every tile is at most 2^{K-1}, so $\sigma(x_t) \le RC \cdot 2^{K-1}$. Since σ grows by at least 2 per move, every game ends within $RC \cdot 2^{K-2}$ moves, whatever the policy. All the policies are proper, and (1)–(2) has a unique solution.

(iii) Let $\mathcal{X}_\sigma = \{x \in \mathcal{X} : \sigma(x) = \sigma\}$. Transitions only go from $\mathcal{X}_\sigma$ to $\mathcal{X}_{\sigma+2} \cup \mathcal{X}_{\sigma+4}$. Hence V^* is computed exactly by a single backward recursion over σ, like the finite-horizon DP algorithm with stage index $\sigma/2$: no value iteration or policy iteration is needed.

Remark (afterstates). A move factors into a deterministic, controlled part $x \mapsto y = M(x,u)$ and a random, uncontrolled part $y \mapsto x'$. The Bellman equation can therefore be written on afterstates, as in (2). This is the afterstate value function of Sutton and Barto [9, §6.8], which the RL part learns instead of action values.

Remark (symmetries). Rotations and reflections of the board (8 for a square board, 4 for a rectangular one) map the problem onto itself, with the moves mapped accordingly, so V^* is invariant under them. The DP does not exploit them and uses them as a correctness test; the n-tuple network of the RL part builds them into the value function.

5 Exact dynamic programming

Replacing the maximisation with the action chosen by a fixed policy gives the exact value of that policy (policy evaluation), which is used for the benchmark policies and, in the RL part, for the learned policies. Ties are broken in the fixed order down, left, right, up. The cost is proportional to $\sum_\sigma |\mathcal{X}_\sigma| \cdot |\mathcal{U}| \cdot 2RC$.

Algorithm 1 Exact DP by stages

Forward pass: starting from the initial distribution (two tiles), generate the reachable sets $\mathcal{X}_\sigma$ for increasing σ
for $\sigma = \sigma_{\max},\ \sigma_{\max} - 2,\ \ldots,\ \sigma_{\min}$ **do** ▷ backward pass
  for each $x \in \mathcal{X}_\sigma$ **do**
    compute $W^*(M(x,u))$ for every $u \in \mathcal{U}(x)$ with (2) ▷ uses V^* on $\mathcal{X}_{\sigma+2} \cup \mathcal{X}_{\sigma+4}$
    $V^*(x) \leftarrow \max_u W^*(M(x,u))$, $\mu^*(x) \leftarrow$ a maximising u ▷ $V^*(x) = 0$ if $\mathcal{U}(x) = \emptyset$
  end for
end for
return μ^* and the value at the start $\sum_x p_0(x)\, V^*(x)$

5.1 Benchmark policies

- **Corner rule**: the first legal move in the order down, left, right, up. It is the classic rule of thumb of keeping the large tiles in one corner and pressing up only when forced.
- **Greedy**: the legal move with the largest immediate score $r(x,u)$, ties broken in the same order.
- **Random**: a uniformly random legal move.

5.2 Size of the state space

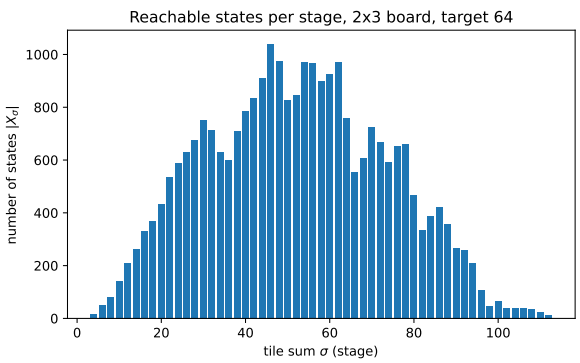
Table 1 reports the number of reachable states, together with the naive bound K^{RC} (each cell holds an exponent in $\{0,\ldots,K-1\}$). Doubling the target roughly doubles the states. Adding cells is far more expensive: with target 64, the 2×4 board has about 30 times the states of the 2×3 board, and the 3×3 board about 180 times. For the original game the bound is $11^{16} \approx 4.6\cdot10^{16}$: exact DP is out of reach, which motivates the RL part. Figure 1a shows how the states of the 2×3 board are distributed over the stages.

Board	Target	$\lvert\mathcal{X}\rvert$	Bound K^{RC}	DP time	P(win)		
				(s)	optimal	corner	greedy
2×2	16	214	256	0.0	0.9625	0.2355	0.2474
2×2	32	422	625	0.0	0.0829	0.0000	0.0000
2×3	32	11 222	15 625	0.2	0.9999	0.2854	0.3365
2×3	64	27 022	46 656	0.3	0.9393	0.0007	0.0013
2×3	128	52 486	117 649	0.6	0.0570	0.0000	0.0000
2×4	64	881 260	1 679 616	100	0.99999	0.3611	0.3295
2×4	128	2 525 754	5 764 801	272	0.9973	0.0044	0.0037
3×3	64	4 820 069	10 077 696	240	1.0000	0.4341	0.4402
3×3	128	17 244 751	40 353 607	–	–	–	–
4×4	2048	intractable	$\approx 4.6\cdot10^{16}$	–	–	–	–

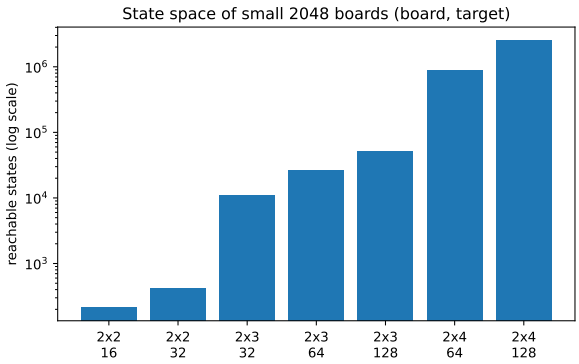
Table 1: Reachable decision states (target not yet created), naive bound, time of the backward recursion in pure Python, and exact win probability of the optimal policy and of two heuristics ($p_4 = 0.1$). The random policy wins with probability 0.1218, 0.0005, 0.0921, 0.0002, 0.0000, 0.0274, 0.0000 and 0.0921 on the first eight boards.

5.3 Results on the 2×3 board with target 64

This board is the reference problem of the project: 27 022 states in 55 stages, solved in a fraction of a second, with an optimal win probability of 0.9393 that is neither trivially 0 nor 1. All the



(a) States per stage, 2×3 board, target 64.



(b) Reachable states of the small boards.

Figure 1: The curse of dimensionality: the number of states grows exponentially with the number of cells.

results below are exact unless stated otherwise.

The heuristics are hopeless. The corner rule wins 0.07% of the games, the greedy rule 0.13% and random play 0.02%, against 93.9% for the optimal policy. On this board the "keep the big tiles in a corner" advice is worthless: the optimal policy plays the corner move in only 49.5% of its decisions (weighted by how often each decision occurs under optimal play). Table 2 lists the departures from the corner rule that matter most, ranked by expected visits times loss of win probability. [comment on the boards: the corner rule refuses to use the free row/the move that keeps the two largest tiles adjacent, etc.]

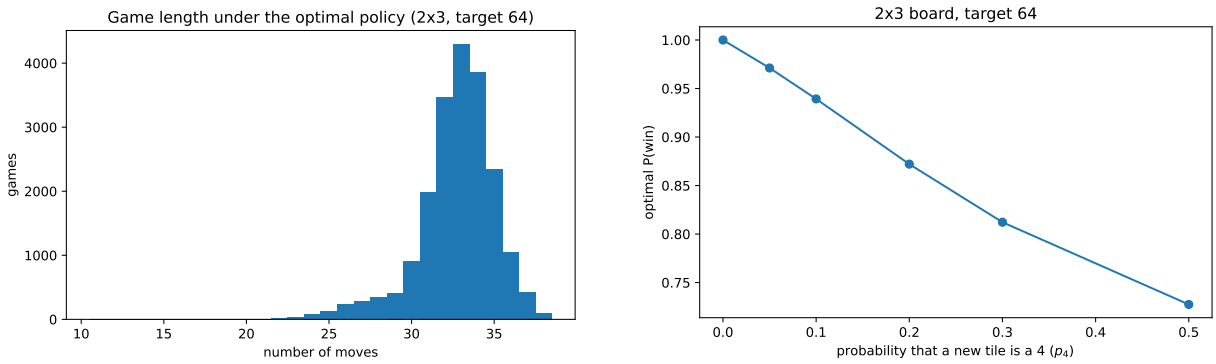
Importance	Board	Corner move	Optimal move	Loss
[from results/dp/table_dp_exceptions.csv]				

Table 2: Most important departures of the optimal policy from the corner rule, 2×3 board, target 64.

Monte Carlo validation. Simulating 20 000 games with the optimal policy gives a win frequency of 0.9395 ± 0.0033 (95% confidence), in agreement with the exact value 0.9393. The games last 32.7 moves on average (Figure 2a).

The role of chance. Figure 2b shows the optimal win probability as a function of p_4. With $p_4 = 0$ (only 2s appear) the game is won with certainty; the 4s are what makes it lose, and at $p_4 = 0.5$ even the optimal player wins only 72.7% of the games. On this small board, therefore, luck decides at least 6% of the games whatever the skill.

Win probability or expected score? The same recursion with the merge points as stage reward maximises the expected score, which is what the game displays and what most 2048 programs optimise. The maximum expected score is 480.3. The score-optimal policy wins 93.87% of the games (against 93.93%) and agrees with the win-optimal move in 97.9% of the decisions. The two objectives are almost, but not exactly, aligned: the score rewards merging, and merging is what makes room for the target. This observation is used in the RL part, where the score is a much denser reward than the win.



(a) Game length under the optimal policy.

(b) Optimal $P(\text{win})$ as a function of p_4.

Figure 2: Monte Carlo check and sensitivity to the spawn distribution, 2×3 board, target 64.

6 Reinforcement learning

6.1 Setting

The agent now interacts only with a simulator of the game, with the interface `reset`/`step` of Gymnasium: it observes the board, chooses a legal move, and receives the next board and a reward. It does not know the spawn distribution. Two reward signals are used:

- **win**: $+1$ when a move creates the target tile, which ends the episode, and 0 otherwise. The return of an episode is the indicator of a win, so the value of a state is its win probability: $v_\pi = P^\pi(\text{win})$ and $v_* = V^*$ of Section 5.3.
- **score**: the points of each move; play continues past the target and the episode ends at game over. The value is the expected score.

By Corollary 1, every episode ends within a bounded number of moves and no state is visited twice in an episode, so the undiscounted return ($\gamma = 1$) is well defined and first-visit and every-visit Monte Carlo methods coincide.

The win reward is sparse: on the 2×3 board with target 64, random play wins 2 games in 10 000, so an agent that explores at random almost never observes a reward. The score reward is dense, and Section 5.3 showed that the score-optimal policy is almost win-optimal. The experiments therefore use both: the score reward to learn, and the exact win probability of the learned greedy policy, computed by the DP, to measure.

Exact learning curves. Because the DP of the small boards is available, the learned policies are not evaluated by simulation but exactly: the greedy policy of the agent is plugged into the backward recursion (policy evaluation) to obtain its win probability and expected score; the learned values are compared with V^* by the root mean square error weighted by the visits of the optimal policy; and the share of decisions (again weighted by the visits of the optimal policy) in which the greedy action is optimal is recorded. The curves have no Monte Carlo noise: all their variability comes from learning.

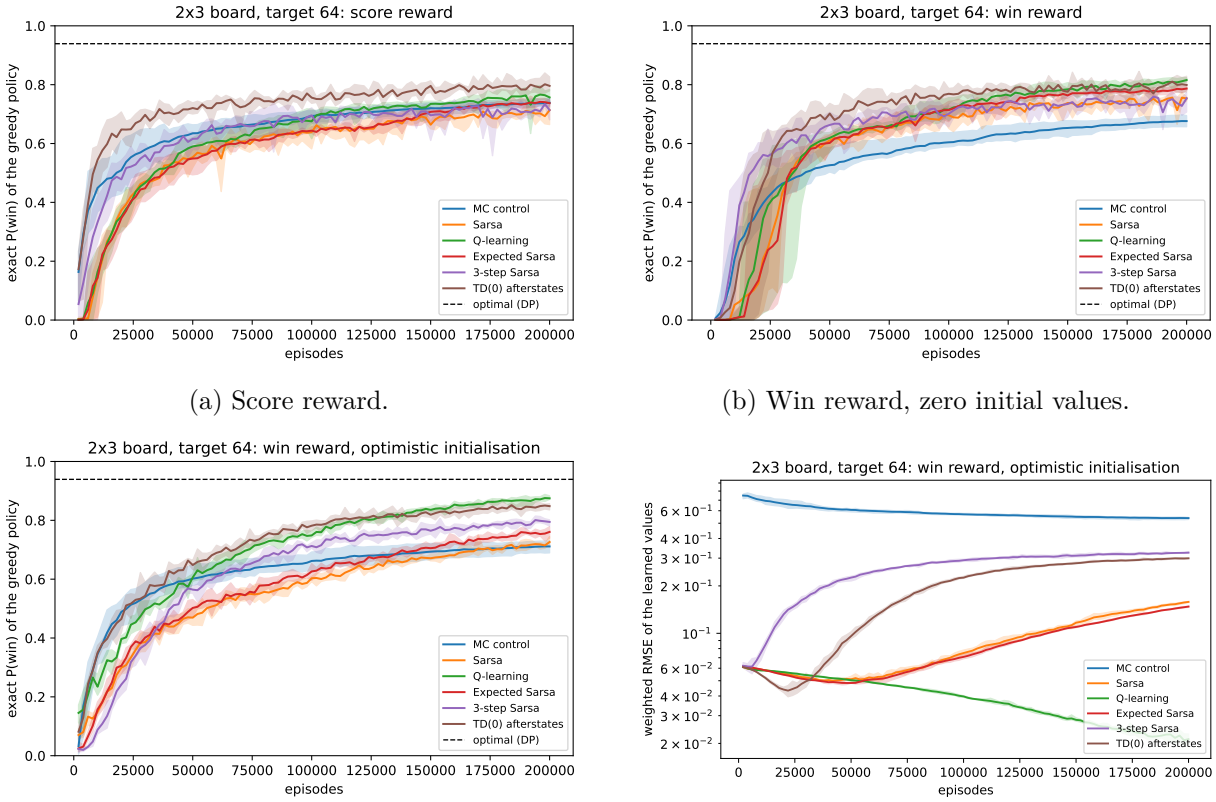
6.2 Tabular methods on the 2×3 board

Six methods of Sutton and Barto [9] are implemented on the tabular action values $Q(x, u)$ (or, for the last one, the afterstate values $V(y)$), with ϵ-greedy exploration restricted to the legal moves:

- **Monte Carlo control** (on-policy, first visit, ϵ-soft; §5.4), with sample-average updates;
- **Sarsa** (§6.4), **Q-learning** (§6.5) and **Expected Sarsa** (§6.6), with constant step size α;

- n**-step Sarsa** (§7.2) with $n = 3$;
- **TD(0) on afterstates** (§6.8): the agent evaluates the afterstate $y = M(x, u)$ of every legal move with the deterministic slide rule, chooses ϵ-greedily by $r(x, u) + V(y)$, and updates $V(y) \leftarrow V(y) + \alpha\,[r' + V(y') - V(y)]$, where (y', r') is the afterstate and reward of the next move. It knows the slide rule but not the spawn distribution. This is the tabular version of the algorithm used on the 4×4 board.

Every method is run for 200 000 episodes with $\alpha = 0.1$, $\epsilon = 0.1$ and 5 seeds, on three settings: score reward; win reward from zero initial values; win reward with optimistic initial values $Q = 1$ ([9, §2.6]). The greedy policy is evaluated exactly every 2 000 episodes. The greedy choice breaks ties in the order down, left, right, up, so an agent with all-zero values plays the corner rule.



(a) Score reward.

(b) Win reward, zero initial values.

(c) Win reward, optimistic initial values.

(d) Value error, win reward, optimistic initial values.

Figure 3: Tabular methods on the 2×3 board, target 64: exact win probability of the greedy policy (mean over 5 seeds, band = min–max) and weighted RMSE of the learned values.

Results. Figure 3 and Table 3 summarise the 90 runs. Every method learns to play far better than any heuristic from the simulator alone: after 200 000 episodes the exact win probability of the greedy policies lies between 0.68 and 0.87, against 0.0007 for the corner rule, and the expected score between 275 and 412 against a maximum of 480. None reaches the optimum 0.9393 within the budget: with constant $\alpha = 0.1$ and $\epsilon = 0.1$ the estimates keep fluctuating and the behaviour policy keeps making exploratory mistakes, which the on-policy methods learn to fear. The ranking is consistent with the theory of the course.

- **Q-learning is the best action-value method in every setting** (0.757, 0.816, 0.875), because it is off-policy: its target $\max_a Q(x', a)$ estimates q_* whatever the exploration, whereas Sarsa and Expected Sarsa estimate the value of the ϵ-greedy policy, which is lower. The error of the values against V^* makes this visible: with the win reward and optimistic initialisation,

the weighted RMSE of Q-learning is 0.021, that of Sarsa 0.158 and of Expected Sarsa 0.148 (Figure 3d), although their greedy policies are not far apart. Expected Sarsa beats Sarsa in every setting (lower variance of the target), and 3-step Sarsa beats both in two of three.

- **TD(0) on afterstates is the best or second best everywhere** (0.796, 0.799, 0.848) and learns fastest (Figure 3a–c) with the fewest parameters: one value per afterstate instead of four per state, 10 478 entries against 12 464 states seen by Q-learning with the score reward. Knowing the deterministic half of the move lets it compare the four moves through their afterstates, which several moves and states share.
- **Monte Carlo control learns fast at first and then slowly**: it leads during the first 20 000 episodes with the score reward, but ends last or second to last, because it does not bootstrap and the returns of 30-move episodes are noisy.
- **The sparse win reward is slower but not hopeless.** From zero initial values the agents play the corner rule at first (the tie-break of the greedy choice), win nothing for about 10 000 episodes, and then take off once the ϵ-greedy exploration has stumbled on a few wins whose value propagates backwards (Figure 3b). Optimistic initial values ($Q_0 = 1$) remove the dead start and leave no state unvisited (the share of the optimal policy's visits to unseen boards drops from 15–20% to 0), and they give the best final results for Q-learning and TD(0) on afterstates. The two on-policy Sarsa methods gain less: under optimism the exploring behaviour is more erratic, and that is what they evaluate.
- **The score reward is a good surrogate**: the greedy policies learned from the score win 71–80% of the games although they never observed a win signal, in line with the 97.9% agreement of the two optimal policies measured in Section 5.3.

[Add the schedule experiment (decaying alpha and epsilon) and the alpha x epsilon grid: results/tabular_grid.]

	score reward			win reward, $Q_0 = 0$			win reward, $Q_0 = 1$		
Method	P(win)	score	agree	P(win)	agree	RMSE	P(win)	agree	RMSE
MC control	0.738	387	0.63	0.676	0.56	0.62	0.711	0.56	0.54
Sarsa	0.713	367	0.53	0.753	0.54	0.68	0.726	0.68	0.16
Q-learning	0.757	383	0.54	0.816	0.56	0.53	0.875	0.78	0.02
Expected Sarsa	0.737	376	0.53	0.787	0.55	0.66	0.761	0.69	0.15
3-step Sarsa	0.714	381	0.59	0.754	0.57	0.62	0.795	0.70	0.33
TD(0) afterstates	0.796	412	0.63	0.799	0.60	0.62	0.848	0.71	0.30
optimal (DP)	0.939	480	1	0.939	1	0	0.939	1	0

Table 3: Tabular methods on the 2×3 board, target 64, after 200 000 episodes ($\alpha = 0.1$, $\epsilon = 0.1$, mean over 5 seeds; the standard deviation over seeds is at most 0.025). Exact win probability and expected score of the greedy policy, share of the optimal policy's decisions in which the greedy action is optimal, and weighted RMSE of the learned state values against V^* (win objective).

6.3 Function approximation: n-tuple networks

On the 4×4 board tabular methods are not feasible. The afterstate value function is approximated linearly with the *n-tuple network* of Szubert and Jaśkowski [10]:

$$\hat{V}(y; \mathbf{w}) = \sum_{m=1}^{M} w_m [\, \phi_m(y) \,],$$

where every feature m is a tuple of n cells of the board, $\phi_m(y) \in \{0, \dots, 15\}^n$ is the pattern of exponents found in those cells, and w_m is a lookup table with 16^n entries. This is a linear approximation with binary features (exactly one entry of each table is active), so the semi-gradient TD(0) update of [9, §9.3] reduces to adding the TD error, divided by M, to the M active entries:

$$w_m[\phi_m(y)] \leftarrow w_m[\phi_m(y)] + \frac{\alpha}{M}[r' + \hat{V}(y') - \hat{V}(y)], \qquad m = 1, \dots, M.$$

The learning algorithm is TD(0) on afterstates of Section 6.2 with the greedy policy ($\epsilon = 0$): the random tile is the only source of exploration, as in [10]. With *symmetric sampling*, every tuple is read in all the images of the board under its 8 symmetries (4 on a rectangular board), all sharing the same table, so that $\hat{V}$ is invariant under the symmetries, like V^*.

Two tuple sets are used: **rows, columns and 2×2 squares** ($n = 4$; on the 4×4 board 5 tables of 16^4 entries after merging the symmetric ones, 327 680 weights, 40 active features), the architecture of [10]; and **2×3 rectangles** ($n = 6$; 2 tables of 16^6 entries, 33.5 million weights, 16 active features), closer to the networks of [12, 5]. On the small boards the tuple set **full board** (one table indexed by the whole afterstate) is the tabular TD(0) of Section 6.2 written as a network, and serves as a sanity check.

6.3.1 Validation on the small boards

[Figures fig_nt_2x3_64_pwin, fig_nt_2x4_128_pwin, fig_nt_3x3_64_pwin and table_ntuple_small.csv. Points: (i) the full-board table converges to the optimum (RMSE to 0) only with optimistic initialisation or exploration: without, the greedy learner locks on the first rewarded moves; (ii) the row/column/square features plateau below the optimum on 2x3 (approximation error of the architecture, quantify), the 6-tuples do better; (iii) on 2x4/128 and 3x3/64 the zero-initialised learner starts as the corner rule (36%, 43%) and climbs to near 1; weighted RMSE; (iv) effect of symmetric sampling.]

6.3.2 The original 4×4 game

[Training grid: tuple sets x alpha x 2 seeds, 300k games each, score reward. Figures fig_4x4_score, fig_4x4_winrate, fig_4x4_maxtile, table_4x4_final.csv. Compare with corner/greedy/random evaluated with the same engine, and with the published results: [10] report winning rate (2048 reached) over 97% after 500k games with the 17 4-tuples; [12] reach 32768 in 31.75% of the games; [4] report 72% for 32768 with optimistic TD learning.]

7 The game itself: what is known

The questions a player asks about 2048 are the questions of this section: can the game always be won, is there an optimal strategy, and what are the good heuristics. The exact results of Section 5.3 answer them completely for the small boards; for the original game the answers come from the literature.

Is 2048 always solvable? Not on the small boards: on the 2×3 board with target 64 even the optimal player loses 6% of the games, and as soon as $p_4 > 0$ there are unlucky spawn sequences that no policy survives (Figure 2b). On the 4×4 board the question is open. Slizkov [8] gives the only rigorous results: a strategy that reaches the 256 tile with probability 1, and a strategy that creates the 2048 tile with probability at least 0.99969. Whether some strategy wins with probability 1 is not known. If the new tiles were placed by an adversary instead of at random, the game would be lost well before 2048: this is easy to believe and to observe with "evil" variants of the game, but we found no proof of it. The largest tile that can ever appear on a 4×4 board is $131072 = 2^{17}$ (sixteen cells holding $2^{17}, 2^{16}, \dots, 2^2$ plus a lucky final 4), and the best programs do reach $2^{15} = 32768$ regularly.

Is the optimal strategy known? No, and there are reasons to believe it is out of reach. The state space of the 4×4 game is bounded by $11^{16} \approx 4.6 \cdot 10^{16}$ states before the target (Table 1), which is beyond exact dynamic programming by many orders of magnitude; the exact solutions of this project stop at 3×3 boards with 5 million states. Computational complexity results show that the difficulty is intrinsic to the rules and not to the board size: Langerman and Uno [6] prove that deciding whether a given position on an $m \times n$ board can be played to reach a given tile value is NP-hard for 2048, Threes! and most of their variants; Abdelkader, Acharya and Dasler [1] prove NP-hardness even when no new tiles appear; Mehta [7] shows that reaching a target configuration is PSPACE-complete for a variant of the game. These are statements about generalised boards and worst-case instances, not about the probability of winning the 4×4 game, but they rule out a simple closed-form optimal strategy.

Which heuristics are good? Human players use the corner rule (keep the largest tile in a corner and the big tiles along one edge, ordered as a "snake"), keep the board monotone and smooth, and keep as many empty cells as possible. Our exact results qualify this advice: on the small boards the corner rule is dramatically worse than optimal play (0.07% versus 93.9% on $2 \times 3/64$, 36% versus 99.999% on $2 \times 4/64$, 43% versus 100% on $3 \times 3/64$), because a rigid move order wastes the few cells available. On the 4×4 board, programs built on exactly these heuristics do well when they are combined with search: the expectimax player of [11], with a hand-tuned evaluation (monotonicity, smoothness, empty cells, merges) and a depth of 4–8 plies, reaches 2048 in essentially every game and 16384 in most. The strongest players learn the evaluation instead of hand-tuning it: temporal-difference learning of n-tuple networks on afterstates [10], the method of Section 6.3, wins 97% of the games after half a million training games; multistage learning and larger tuples [12] reach the 32768 tile in 31.75% of the games; Jaśkowski [5] adds temporal coherence learning, weight promotion and redundant encoding, and [4] optimistic initialisation, to reach 32768 in about 72% of the games with average scores above 600 000. All of these are, at the core, the semi-gradient TD(0) of this project: the state of the art of 2048 is a course method applied with care and a lot of computation.

8 Discussion and conclusions

[To write once all results are in. Points: what the exact values say about the heuristics; the DP as a tool to validate RL (exact learning curves); which RL methods worked, why (dense reward, exploration, afterstates halve the branching); the approximation error of n-tuple networks measured exactly on small boards; the 4x4 results versus the literature; what remains (luck bounds the achievable win probability; the optimal policy of the real game is unknown).]

A Reproducibility

The code is at [GitHub URL]. All the numbers and figures of this report are produced by the scripts in `experiments/` from the modules `game2048.py` (rules, Gymnasium-style environment), `dp2048.py` (exact DP), `fast2048.py` (bitboard engine compiled with numba, checked move by move against the reference engine by `test_2048.py`), `rl2048.py` (tabular methods, exact evaluator) and `ntuple2048.py` (n-tuple network). [list the commands]

References

[1] Ahmed Abdelkader, Aditya Acharya, and Philip Dasler. 2048 without new tiles is still hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 1:1–1:14, 2016. Preprint: On the complexity of slide-and-merge games, arXiv:1501.03837.

[2] Dimitri P. Bertsekas. *Reinforcement Learning and Optimal Control*. Athena Scientific, 2019.

[3] Gabriele Cirulli. 2048. `https://github.com/gabrielecirulli/2048`, 2014.

[4] Hung Guei. *On Reinforcement Learning for the Game of 2048*. PhD thesis, National Yang Ming Chiao Tung University, 2022. arXiv:2212.11087.

[5] Wojciech Jaśkowski. Mastering 2048 with delayed temporal coherence learning, multistage weight promotion, redundant encoding, and carousel shaping. *IEEE Transactions on Computational Intelligence and AI in Games*, 10(1):3–14, 2018. arXiv:1604.05085.

[6] Stefan Langerman and Yushi Uno. Threes!, Fives, 1024!, and 2048 are hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 22:1–22:14, 2016. Journal version: Theoretical Computer Science 748, 17–27, 2018.

[7] Rahul Mehta. 2048 is (PSPACE) hard, but sometimes easy, 2014. arXiv:1408.6315.

[8] Alexey Slizkov. Computational bounds for the 2048 game, 2023. arXiv:2303.07266.

[9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[10] Marcin Szubert and Wojciech Jaśkowski. Temporal difference learning of N-tuple networks for the game 2048. In *IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, 2014.

[11] Robert Xiao. 2048 AI: expectimax optimisation with a bitboard engine. `https://github.com/nneonneo/2048-ai`, 2014.

[12] Kun-Hao Yeh, I-Chen Wu, Chu-Hsuan Hsueh, Chia-Chuan Chang, Chao-Chin Liang, and Han Chiang. Multistage temporal difference learning for 2048-like games. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(4):369–380, 2017.